The Date Crew | Product Growth Assessment

Krishnasinh Jadeja · github.com/K-Jadeja · jadejakrishna42@gmail.com

Watch the 80-second version: https://k-jadeja.github.io/tdc-before-you-send/ · Try the prototype: https://k-jadeja.github.io/tdc-before-you-send/prototype.html · Code: https://github.com/K-Jadeja/tdc-before-you-send

Two words I use throughout. The **client** is the person you are finding a partner for. The **match** is the person whose profile you send them.

One reading I lean on: I take "contact details shared" to mean the match also said yes. If that is wrong, tell me. It changes my third check and my headline number.

Part 1 - Diagnose the Problem

Stage	Number	From previous step	Lost at this step
Profiles shared	1,000		
Profiles accepted	310	31%	690
Contact details shared	210	68%	100
Conversations started	150	71%	60
Meetings fixed	75	50%	75
Meetings completed	42	56%	33

1. Three questions I would ask first

1. **What happens between "accepted" and "contact details shared"?** A hundred client yeses stop there. If the match said no, an email only works when two people say yes: 210 times in 1,000. If not, I have misread your funnel.
2. **Why do profiles go out that break what the client already told us?** That is 35% of 690 rejections, about 242 emails, one in four. What does the matchmaker have in front of her when she chooses? Hard-to-see preferences, a thin pool, or deliberate stretching would each need a different fix. B converts half as often as A (21% against 44%), so I would check whether most of the 242 are B's.
3. **When a client says no, then yes to someone very similar, what changed?** If nothing did, a vague no is not a verdict, and those people deserve a second look.

2. The biggest problem

The moment of sending. 790 of 1,000 emails did not end in two yeses. The clearest slice is the 242 that went against a stated preference. If search time tracks emails sent, that is about half an hour of each client's two hours a week.

Assumptions: the 35% is labelled correctly, and later steps hold. Replace those 242 with profiles that fit and convert like the other 758 (210 in 758, 27.7%), and you get about 67 more mutual yeses and 13 more meetings (42 to 55).

Runner-up: 44% of fixed meetings never happen. A similar prize, but the brief gives no clue why. (If it is plain no-shows, a confirmation the day before and an easy way to reschedule is a one-day fix. I would ask before building.)

3. Three metrics

1. **Emails that go against a stated preference** (24% today). It moves within days.
2. **Two yeses per 100 emails** (21 today). Client acceptance alone can rise while this falls.
3. **Completed meetings per client per month.** What clients pay for, and it stops us "improving" by sending less.

Part 2 - Design a Solution

Problem

One profile in four goes out against something the client already said. I do not yet know what the matchmaker has in front of her when she chooses, and that is the first thing I would ask. Each of those emails costs search time, uses up a profile for that client, and tells a paying client we were not listening.

User

The matchmaker, at the moment she is about to press Send. The tool has to read like a colleague's note.

Solution

Cheapest first.

1. **This week, no code.** Pin each client's deal breakers at the top of their email thread, then count for a week how many emails break one. If that fixes it, I build less.
2. **A side panel in Gmail.** Your email already runs on Gmail, so there is nothing new to open. Beside the draft, it answers three questions:
   - *Does this profile break one of the client's deal breakers?* If yes, it says don't send. Softer preferences only get a warning.
   - *Has the client refused someone like this before?* A no without a reason is often not final, so the panel also lists people worth a second look.
   - *Will the match say yes back?* If three clients said yes to someone and that person declined all three, ask them first. The client never hears about it. This one is a hypothesis. It depends on my reading of your funnel and needs months of real history.

   There is no score. She decides, and can always send anyway. If matchmakers already search inside an internal tool, the same checks belong there too; Gmail is the last place to catch a miss.
3. **Not yet: learning each client's "type".** This was my favourite idea. I tested it before building, and it lost (see Evidence).

Data

- Each client's preferences, with deal breakers marked

- Profile details
- A record of who was sent to whom, and what each person said

That record lives in email today. Building it is the first job.

Technology

- A Gmail add-on or a Chrome extension for the panel (which one is a week-one decision)
- The Gmail API to read threads
- An LLM with a fixed output format to turn replies into rows in the record, checked against 200 hand-labelled replies
- The three checks are plain rules with tests. No model picks anyone.

Two weeks

- **Days 1-2:** sit with A and B while they work, and ask the funnel question. Count three things in real data: emails against a stated preference, by matchmaker; how often each match says yes back; vague rejections later followed by a yes to someone similar.
- **Days 3-7:** the record and the first check, replayed over last month.
- **Week 2:** the other two checks, but only if the counts support them. Then a pilot on half of each matchmaker's clients, so the gap between A and B does not blur the result.

Success metric

Two yeses per 100 emails, from 21 to 25 within a month. Removing every email that goes against a stated preference would give 27.7, which is the ceiling from this fix alone. The early signal is that share of emails falling from 24% toward 5%. Guardrail: mutual yeses per client per week must not fall.

Part 3 - Prototype

https://k-jadeja.github.io/tdc-before-you-send/prototype.html (code: https://github.com/K-Jadeja/tdc-before-you-send)

A mock-up of the Gmail side panel. You play the matchmaker: choose a client, choose who to introduce, read the panel, press Send, and the client's reply comes back. Every person and email in it is invented to match your numbers, so it shows how the tool would behave, not that it works.

Part 4 - One Curveball

The number is two yeses per 100 emails, and it has not moved.

First: is the panel being used? How many emails were written with it open, and how often its "don't send" was overridden. If matchmakers work around it, the idea has not been tested yet.

Then the data:

- Emails against a stated preference. If those fell and two-yeses did not, deal breakers were not the lever, or the replacements were no better.
- The reasons given for overrides. "Nobody else to send" means the pool is the real limit.

- "Ask them first": was it followed, and did the match answer the way their history suggested?
- Fifty replies labelled by hand, to check the record itself.

Decision:

- *Iterate* if it is use or data quality: put the check into the Send step so it cannot be skipped.
- *Change the solution* if it is the pool: the problem becomes finding more people.
- *Kill it* if emails that fit the client's preferences get two yeses no more often than ones that do not. Then stated preferences are not the lever.

One more two-week cycle, then decide.

Why it looks like this

Before designing anything I read how you describe yourselves, and it changed the design.

- **Your site says "No AI. No App. No Endless Swiping."** So nothing here faces the client, nothing is scored, and the matchmaker is never overruled. The only AI works behind the scenes, reading replies into a record. The checks are plain rules, and she can see the reason for each one.
- **Your job post asks for AI only where it earns its place.** That is why my first step is a pinned note and a week of counting, not a build.
- **I dropped two ideas for the same reason.** One was a set of sample profiles for new clients to say yes or no to, which is swiping by another name. The other was learning each client's "type" from photos. The model could see it, but the choice data said it would not help yet.
- **People are not their forms.** In the speed-dating test, what someone said they wanted predicted their choices no better than a stranger's list. And a no given to a human matchmaker may be polite rather than final. So the panel treats deal breakers as firm and a vague no as an open question.
- **Matchmakers are craftspeople.** A tool that grades them gets ignored. This one hands her what is on record and gets out of the way.

Evidence: what I tested before choosing

I had none of your data, so I used a public one. Both tests are in `analysis/` in the repo.

1. Can you learn a person's "type" from their choices? A Columbia University speed-dating study: 551 people, 8,368 yes-or-no decisions. I tried to rank each person's later dates using only what was known earlier. The score is how often a yes is ranked above a no; 50% is a coin flip.

Predict from	Score
What the person said they want	74.7%
A random stranger's wish list	74.1%
The person's own earlier choices (about 8)	68.0%
What most people respond to	77.4%

A person's wish list did no better than a stranger's. A handful of their own choices did worse than general appeal. That is what parked the "type" idea.

The same data showed two more things. The same person gave near-identical partners different answers 33% of the time. And 63% of yeses were never returned, with the most wanted people being the pickiest.

2. Can a small on-device model see what is in a profile photo? EmbeddingGemma 2, run locally on 52 generated faces. From three yeses and three nos it found a preference for glasses 96% of the time, long hair (among women) 78%, a broad smile 62%. It works, but the first test says not to build on it yet.

Limits. Speed dating between students in 2002-04 is not matchmaking. I ran each test once. They tell me direction, not size, and my first two days would repeat them on your data.

AI Usage

- I used Claude Code for all of it: reading the brief, both tests, the prototype, the film, and first drafts of this document.
- Claude's first move was to pick the 35% and build a preference checker in one pass, without asking me. I set that aside and asked what clients are not telling us.
- I then pushed for learning each client's "type" with an on-device model. Claude suggested testing it on public data first. It lost, so I parked it.
- The brief suggests 60 to 90 minutes. The first answer took about that. I then spent an evening on it, because I did not trust my own favourite idea.